TECHNICAL ARCHITECTURE & PROJECT REPORT

Smart Campus Issue Reporting & Maintenance Management Platform

Comprehensive Module-by-Module Breakdown: Operational Workflows, Internal Mechanisms, Software Architecture, and Technology Stacks.

CORE ARCHITECTURE

Client-Server REST SPA (Micro-Service Ready)

FRONTEND ENGINEERING

React 19, TypeScript, Vite 8, Tailwind CSS v4

BACKEND ENGINEERING

Node.js, Express, TypeScript, TSX Watch

DATA PERSISTENCE

MongoDB 8.0+, Mongoose ODM Schema Modeling

SECURITY & AUTHENTICATION

Institutional Dataset Verification, JWT (7d), BcryptJS

AUTOMATED WORKFLOWS

Real-time Duplicate Prevention & 10-Min SLA Cron Engine

Project: Campus Issue Reporting & Resolution System

Institution: Adhiyamaan College of Engineering

Date: October 2026

1 High-Level System Architecture & Topology

The **Smart Campus Issue Reporting & Maintenance Management Platform** is engineered to eliminate traditional campus maintenance delays, paper-based slips, and uncoordinated facility repairs. It connects four distinct institutional stakeholders (**Students**, **Faculty**, **Maintenance Technicians**, and **Campus Administrators**) through a unified, reactive digital portal.

Multi-Tier Component Topology

The system follows a modern decoupled 3-Tier Web Architecture:

`TIER 1 : CLIENT` **Presentation Layer (React 19 SPA):** Hosted via Vite on port 3000. Features role-driven reactive dashboards, contextual modals, live camera video capture, multi-format file uploads, Web Speech API speech-to-text voice input, and confetti completion triggers.

`TIER 2 : SERVER` **Application / API Gateway (Node.js & Express):** Runs on port 5000 with CORS, JSON body parsers, disk storage Multer engine, JWT authorization filters, database-level RBAC role validators, and an automated background SLA escalation scheduler.

`TIER 3 : DATABASE` **Data Persistence Layer (MongoDB & Mongoose):** Structured document collections including Users, Approved Campus Members (Excel dataset), Locations, Categories, Departments, Complaints with embedded lifecycle timelines, and immutable Administrative Audit Logs.

Institutional Role-Based Access Control (RBAC) Matrix

Every API route is secured at the server level by verifying the caller's true database record, ignoring any client-side claims.

User Role	Identity Identifier	Account Creation Route	Authorized Capabilities & Permitted Actions
STUDENT	Register Number (e.g. `6176AC21UCS143`)	Public Identity Verification against College Excel records	Report campus issues, view own tickets, community upvote recurring faults, track repair timelines, review resolved repairs with 1-5 star ratings.
FACULTY	Faculty ID (e.g. `FAC-CSE-001`)	Public Verification against College Excel or Admin Provisioned	Priority classroom & lab issue reporting, department ticket tracking, resolution verification.
STAFF (TECHNICIAN)	Staff ID / Email (e.g. `STAFF-ELEC-001`)	Admin Provisioned	Department task queue viewing, accept task, start work (In Progress), upload mandatory post-repair proof photo

User Role	Identity Identifier	Account Creation Route	Authorized Capabilities & Permitted Actions
			(`afterImages`), submit completion notes.
ADMIN	Admin ID / Email (e.g. `admin@campus.edu`)	System Seed / Master Admin	Master complaint dispatch center, technician assignment, priority escalation, user management (activate/deactivate), faculty/staff provisioning, system audit logs, SLA breach monitoring.

2 Module 1: Authentication & Institutional Identity Verification

Module Objective & Overview

Prevents unauthorized public spam, spoofed identities, or malicious account creation by strictly verifying every user against an authorized institutional dataset before granting access to create an account.

1. Technologies & Libraries Employed

`XLSX (SheetJS)` `BcryptJS (10 Salt Rounds)` `JSON Web Tokens (JWT)` `Mongoose ODM` `React Context API`

2. Operational Mechanism & Step-by-Step Flow

STEP 1 **Excel Dataset Ingestion:** The backend script (`importCampusData.ts`) parses the college's official roster (`smart_campus_login_dataset.xlsx`) using `xlsx.readFile()` , extracting Register Numbers, Names, Departments, Batches, and Dates of Birth (DOB) into the `ApprovedCampusMember` MongoDB collection.

STEP 2 **Two-Factor Identity Verification (`POST /api/v1/auth/verify-identity`):** When a student or faculty registers, they submit their Register Number/Faculty ID, DOB, and Role. The server normalizes the date via `formatDob()` (handling SSF serial date codes, YYYY-MM-DD, and DD-MM-YYYY formats) and matches it against the approved collection.

STEP 3 **Secure Account Creation (`POST /api/v1/auth/register`):** Once verified, the user chooses their personal password. The server hashes the password with `bcrypt.genSalt(10)` and `bcrypt.hash()` , creates the permanent `User` record, flags `accountCreated = true` on the roster, and returns a signed JWT.

STEP 4 **Multi-Identifier Login (`POST /api/v1/auth/login`):** Users can log in using either their Register Number, Faculty ID, or Email. The server compares hashes via `bcrypt.compare()` , checks account active status (`isActive: true`), and generates a 7-day signed JWT containing user claims.

STEP 5 **Server-Side RBAC Guard (`authMiddleware.ts`):** The `authenticateToken` middleware validates the JWT header (`Bearer <token>`) using `jwt.verify()` . Crucially, it queries MongoDB directly (`User.findById()`) on every request to verify the user's live role and active status, mitigating token tampering attacks.

3 Module 2: Complaint Reporting & Multimodal Evidence Submission

Module Objective & Overview

Enables campus stakeholders to file rich, highly descriptive issue reports within 30 seconds, capturing accurate location data, image/video proof, real-time voice speech, and automated duplicate prevention.

1. Technologies & Libraries Employed

`HTML5 MediaDevices API` `Web Speech Recognition API` `Multer Multipart Storage` `Lucide React Icons` `Canvas Confetti`

2. Operational Mechanism & Key Features

FEATURE A **Hierarchical Location Selection:** Pre-structured campus topology (`PREDEFINED_LOCATIONS`) maps Building Blocks (Main, MBA, Arch, Civil, Canteen, Library) → Floors (1st to 4th) → Specific Rooms, Computer Labs, Studios, or Restrooms.

FEATURE B **Real-Time Duplicate Issue Detection:** As soon as a user selects Building + Floor + Room, the client triggers `GET /api/v1/complaints/check-duplicate` . The server queries active tickets (excluding Closed/Verified) at that exact coordinate. If an issue is already open, the user is alerted and can simply *Upvote* the ticket rather than clogging the maintenance queue.

FEATURE C **Multimodal Proof Capture (Live Camera & File Upload):**

- **Live Camera Capture:** Invokes `navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } })` to open the user's camera, stream video into a live canvas, and capture instant snapshots.
- **Multer File Upload (`POST /complaints/upload`):** Handles multipart/form-data with disk storage, generating cryptographically unique filenames (timestamp + random suffix) for image and video formats (JPG, PNG, MP4, MOV up to 50MB).

FEATURE D **Real-Time Speech-to-Text Voice Transcription:** Uses the browser's native `webkitSpeechRecognition` engine. Users can dictate descriptions hands-free in either **English (en-US)** or **Tamil (ta-IN)**. Live interim transcripts are transcribed into the description box with an active recording duration timer.

FEATURE E **Privacy-Guaranteed Anonymous Reporting:** Users can toggle `isAnonymous: true` . The server sanitization layer (`sanitizeComplaint()`) strips the reporter's personal ID, name, and contact details for technician and public views, displaying only `"Anonymous Reporter"` while allowing the author and admin to view necessary details.

4 Module 3: Ticket Lifecycle, SLA Engine & Auto-Escalation

Module Objective & Overview

Maintains an auditable, rigorous state machine for every ticket from initial submission to final verification, enforcing Service Level Agreements (SLAs) with background cron escalation.

1. Technologies & Libraries Employed

`Node.js setInterval Worker` `Mongoose Subdocuments & Timelines` `MongoDB Query Operators ($lt, $in)`

2. Complaint Lifecycle State Machine

State / Status	Trigger Event	Responsible Actor	Next Permitted States
`Reported`	Complaint submitted by user	Student / Faculty	`Assigned`
`Assigned`	Allocated to Department & Staff	Admin	`Accepted`, `In Progress`
`Accepted`	Technician acknowledges assignment	Technician Staff	`In Progress`
`In Progress`	Technician starts repair work on site	Technician Staff	`Resolved`
`Resolved`	Technician uploads proof photo & notes	Technician Staff	`Closed`, `In Progress` (Re-open)
`Closed`	Reporter verifies repair & rates service	Original Reporter	Terminal State (Final)

3. SLA Calculation & Automated Background Escalation

SLA RULES When a ticket is raised, the category configuration determines its SLA target date: `slaDueDate = Date.now() + (category.defaultSLAHours * 3600 * 1000)`.
Examples: *Electrical Hazards: 4 hours* | *Sanitation: 8 hours* | *Water Leakage: 12 hours* | *Civil & Furniture: 48 hours*.

CRON WORKER

Background Escalation Worker (`escalationService.ts`): Triggered on server startup and every **10 minutes** via `setInterval()`:

1. Queries: `status IN ['Reported', 'Assigned', 'Accepted', 'In Progress'] AND isEscalated == false AND slaDueDate < now`.
2. Automatically sets `isEscalated = true` and bumps priority to `Urgent`.
3. Appends an automated audit entry to the ticket timeline: *"SLA target date breached. Ticket escalated to Urgent priority for immediate management review."*

5 Module 4 & 5: Technician Dispatch & Two-Way Verification Closing Loop

Module 4: Technician Execution Portal

Empowers maintenance workers with a lightweight mobile-friendly interface to manage assigned campus tasks without bureaucratic paperwork.

KANBAN TABS Technicians view tasks filtered by their specific department (Electrical, Plumbing, IT, Civil, Sanitation) segmented into: **Assigned**, **In Progress**, and **Resolved**.

PROOF OF WORK **Mandatory Photographic Evidence (`POST /complaints/:id/resolve`):** Technicians cannot mark an issue resolved with text alone. The API strictly requires an `afterImages` array (photographic proof of repair) and resolution notes before transitioning the ticket to `Resolved` .

Module 5: Student Verification & Feedback Closing Loop

Ensures accountability by placing final sign-off power in the hands of the person who reported the problem.

FLOW A **Satisfied & Closed (`POST /complaints/:id/verify` with `isSatisfied: true`):** The reporter inspects the before/after photos and on-site fix, awards a 1 to 5 star rating with optional review comments, and closes the ticket. Confetti animations celebrate the successful resolution!

FLOW B **Dissatisfied & Re-opened (`isSatisfied: false`):** If the issue was improperly repaired, the reporter rejects the resolution. The system immediately rolls the status back to `In Progress` , notifies technicians with the rejection remarks, and resets the verification state.

6 Module 6: Administrator Command Center & Auditing

Module Objective & Overview

Provides university executives, estate managers, and maintenance directors with comprehensive operational control, analytics, user provisioning, and compliance auditing.

1. Technologies & Libraries Employed

`MongoDB Aggregation Pipeline ($lookup, $group, $project)` `CSV Data Exporter` `Immutable AuditLog Schema` `Express Role Middleware`

2. Core Capabilities & Architectural Mechanics

ANALYTICS

Real-Time KPI Dashboard (`GET /admin/analytics/overview`): Aggregates metrics across the campus database using MongoDB Aggregation Pipelines:

- Total complaints, active vs resolved ratio, escalated SLA breaches.
- Category breakdown (pie/bar data grouped by `categoryName`).
- Building hotspot breakdown (e.g. Main Block vs Arch Block).
- Staff workload analytics: calculates active tickets vs total completed per technician using `$lookup` and `$filter` .

DISPATCH

Master Dispatch Queue & Assignment (`PATCH /complaints/:id/assign`): Administrators can re-assign departments, allocate specific technician staff members, and manually override priority levels (e.g. Low to Urgent).

USERS

User Directory & Privileged Provisioning (`GET/POST/PATCH /admin/users`):

- Search and filter campus users by role (Student, Faculty, Staff, Admin) and department.
- Instant activation/deactivation toggle (`isActive: true/false`) to revoke access.
- Provision new Faculty and Staff accounts with mandatory institutional domain verification (`@adhiyamaan.ac.in`).

AUDIT LOG

Immutable Compliance Audit Log (`AuditLog` Collection): Every sensitive administrative action (`ROLE_CHANGE` , `USER_ACTIVATION` , `USER_DEACTIVATION` , `USER_CREATION`) records: `action` , `performedBy` (Admin ID & Name), `targetUser` , `timestamp` , and exact delta parameters in `details` .

7 Technology Stack & Architecture Summary Matrix

Architectural Layer	Technology / Framework	Specific Version	Primary Role & Responsibilities
Frontend UI Framework	React	`^19.2.7`	Component-driven single page user interface and state management.
Frontend Build Tool	Vite	`^8.1.1`	Lightning-fast development bundler, HMR, and proxy configuration.
Type System	TypeScript	`~6.0.2 / 5.7.2`	Strict compile-time type safety across client and server entities.
CSS & Styling	Tailwind CSS	`^4.3.3`	Modern utility-first styling with modern responsive tokens.
Iconography	Lucide React	`^1.25.0`	Accessible, consistent UI iconography across dashboards.
HTTP Client	Axios	`^1.18.1`	Promise-based HTTP client with Bearer token interceptors.
Backend Runtime	Node.js & Express	`^4.21.2`	REST API routing, middleware chaining, and endpoint handling.
Database Engine	MongoDB (Local/Atlas)	`^8.0+`	Document storage for users, complaints, locations, and audit logs.
ODM Framework	Mongoose	`^8.9.3`	Schema definitions, validation rules, indices, and aggregations.
Authentication Token	JSON Web Token (JWT)	`^9.0.2`	Cryptographically signed stateless authentication tokens.
Password Security	BcryptJS	`^2.4.3`	One-way password salting and hashing (10 rounds).
File & Media Storage	Multer	`^1.4.5-lts.1`	Disk storage handler for multi-format photos and video proof.
Dataset Parser	XLSX (SheetJS)	`^0.18.5`	Parses official campus Excel rosters for student/faculty verification.

Conclusion & Viva Summary

The **Smart Campus Issue Reporting & Maintenance Management Platform** solves the critical campus facility bottleneck by combining **institutional data verification**, **frictionless multimodal reporting** (photo, video, and Tamil/English voice dictation), **duplicate mitigation**, and an **automated 10-minute SLA breach escalation worker**. With full end-to-end auditability and student sign-off, it transforms campus operations into an accountable, transparent, and modern digital workflow.

Smart Campus Issue Reporting & Maintenance Management Platform — Comprehensive Technical Report